

Parte 2: Documento Técnico

1. Introducción

El presente documento describe el análisis, diseño e implementación de una prueba de concepto (PoC) para la integración entre un Sistema de Gestión, un Sistema Contable y un Sistema de Lealtad.

El problema principal identificado se relaciona con la necesidad de procesar las ventas de manera confiable sin acoplar directamente el Sistema de Gestión con el Sistema Contable. La solución debe contemplar indisponibilidades temporales de los sistemas externos, evitar la pérdida o duplicación de facturas, respetar la ventana de cierre contable y garantizar que los puntos de lealtad sean acreditados únicamente después de la emisión correcta de una factura.

Para resolver estos requerimientos se propone una Capa de Integración basada en procesamiento asíncrono mediante RabbitMQ. La implementación utiliza Python y FastAPI para los servicios, RabbitMQ como mecanismo de mensajería persistente, Docker Compose para la orquestación del entorno y Pytest para las pruebas automatizadas.

La solución incorpora mecanismos de resiliencia como reintentos con backoff exponencial, idempotencia, Dead Letter Queue (DLQ), control del cierre contable y logging estructurado para trazabilidad.

2. Backlog priorizado

El backlog fue priorizado considerando principalmente la continuidad operativa, la integridad de la información contable y la consistencia entre los sistemas involucrados. Las historias relacionadas con facturación, trazabilidad y cierre contable presentan impacto alto debido a que un fallo puede provocar pérdida o duplicación de facturas, inconsistencias contables o incumplimiento de las reglas de negocio.

3. Riesgos técnicos

R1 – Indisponibilidad del Sistema Contable. Probabilidad media e impacto alto. Como mitigación, la arquitectura incorpora una cola persistente y una política automática de reintentos.
R2 – Duplicación de facturas debido a reintentos. Probabilidad baja e impacto medio. Se mitiga utilizando id_venta como identificador de referencia y un mecanismo de idempotencia que impide volver a emitir una factura previamente creada.
R3 – Error en la gestión del cierre 22:00–23:59. Probabilidad media e impacto alto. Se mitiga mediante una validación explícita del horario de cierre antes de invocar al Sistema Contable. Durante esta ventana los mensajes permanecen pendientes para su procesamiento posterior.

4. Arquitectura AS-IS

La arquitectura AS-IS presenta un flujo predominantemente síncrono entre el Sistema de Gestión, el middleware y el Sistema Contable. Esta dependencia provoca que una

indisponibilidad del Sistema Contable pueda impedir completar correctamente el proceso de facturación.

Asimismo, no existe un mecanismo explícito de cola persistente y reintentos que garantice la conservación de las solicitudes ante una interrupción. Esto genera dependencia de procesos posteriores de reconciliación y aumenta el riesgo de facturas pendientes o inconsistencias entre sistemas.

5. Arquitectura TO-BE

La arquitectura TO-BE desacopla la recepción de ventas de la facturación mediante una cola persistente. La Capa de Integración recibe los eventos y los almacena para procesamiento asíncrono.

Durante la ventana de cierre entre las 22:00 y las 23:59 no se envían nuevas solicitudes al Sistema Contable. Una vez finalizada la ventana, los mensajes pendientes pueden continuar su procesamiento.

La notificación al Sistema de Lealtad se realiza únicamente después de que el Sistema Contable haya confirmado la emisión de la factura.

6. Arquitectura implementada

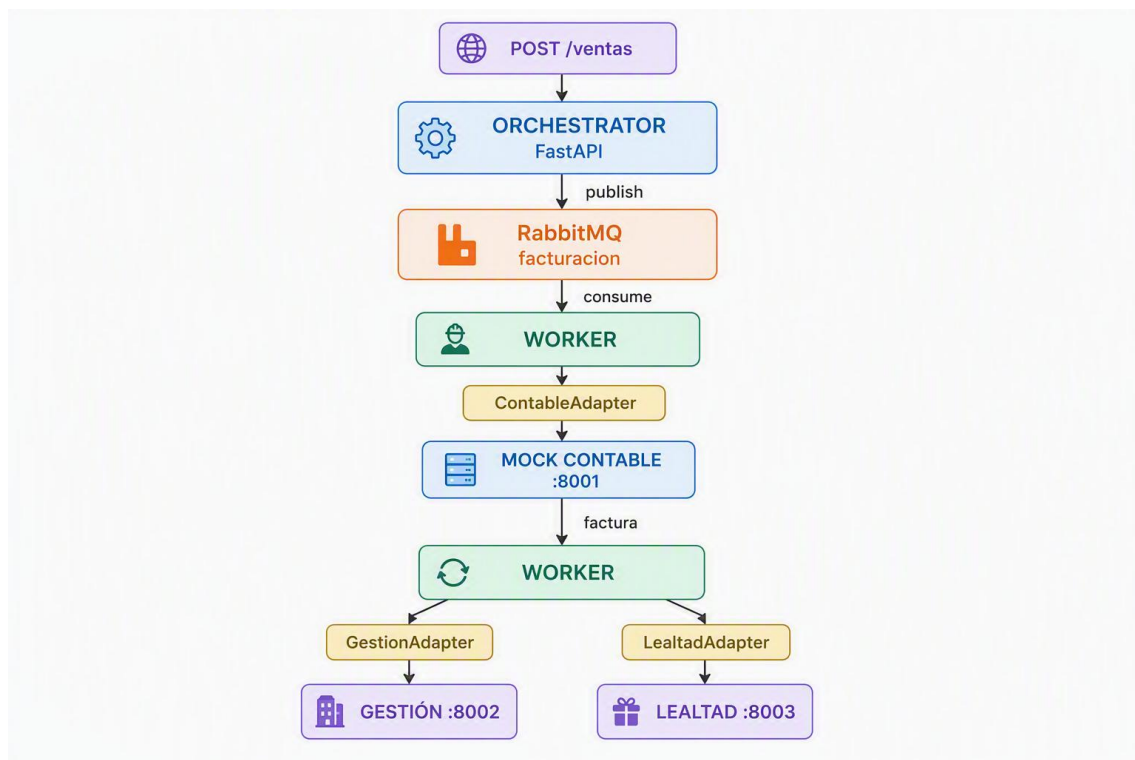


Ilustración 1 - Autoria Propia

La PoC fue implementada utilizando una arquitectura orientada a eventos. El Orchestrator expone POST /ventas, valida la solicitud y publica el evento en RabbitMQ.

Un Worker independiente consume los mensajes de la cola facturación y ejecuta la integración con los sistemas externos mediante adaptadores.

Los tres sistemas externos fueron representados mediante servidores mock desarrollados con FastAPI: Sistema Contable en el puerto 8001, Sistema de Gestión en el 8002 y Sistema de Lealtad en el 8003.

Todos los componentes son desplegados mediante Docker Compose, permitiendo reproducir la solución completa en un entorno controlado.

7. Validación del JSON

```
{
  "id_venta": "V-10001",
  "id_cliente": "CLI-1001",
  "items": [
    {
      "sku": "SKU-001",
      "descripcion": "Tiempo estacion gamer",
      "cantidad": 1,
      "precio_unitario": 15.0
    }
  ],
  "total": 15.0,
  "canal": "ecommerce",
  "timestamp": "2026-08-17T01:00:00-05:00"
}
```

8. Patrón Adapter

Se utilizó el patrón Adapter para encapsular la comunicación con los sistemas externos. De esta forma, el Worker no necesita conocer directamente los detalles de los endpoints HTTP de cada sistema.

ContableAdapter encapsula la emisión de facturas, GestionAdapter la confirmación de la factura y LealtadAdapter la acreditación de puntos.

Esta decisión reduce el acoplamiento y permite sustituir los mocks por servicios reales sin modificar sustancialmente la lógica principal de procesamiento.

9. Cola persistente

RabbitMQ implementa el mecanismo de desacoplamiento y persistencia de eventos. La cola principal utilizada es facturacion, declarada como durable, y los mensajes se publican utilizando entrega persistente.

El Worker utiliza prefetch_count=1, permitiendo un procesamiento secuencial de los mensajes en esta PoC.

10. Cierre contable

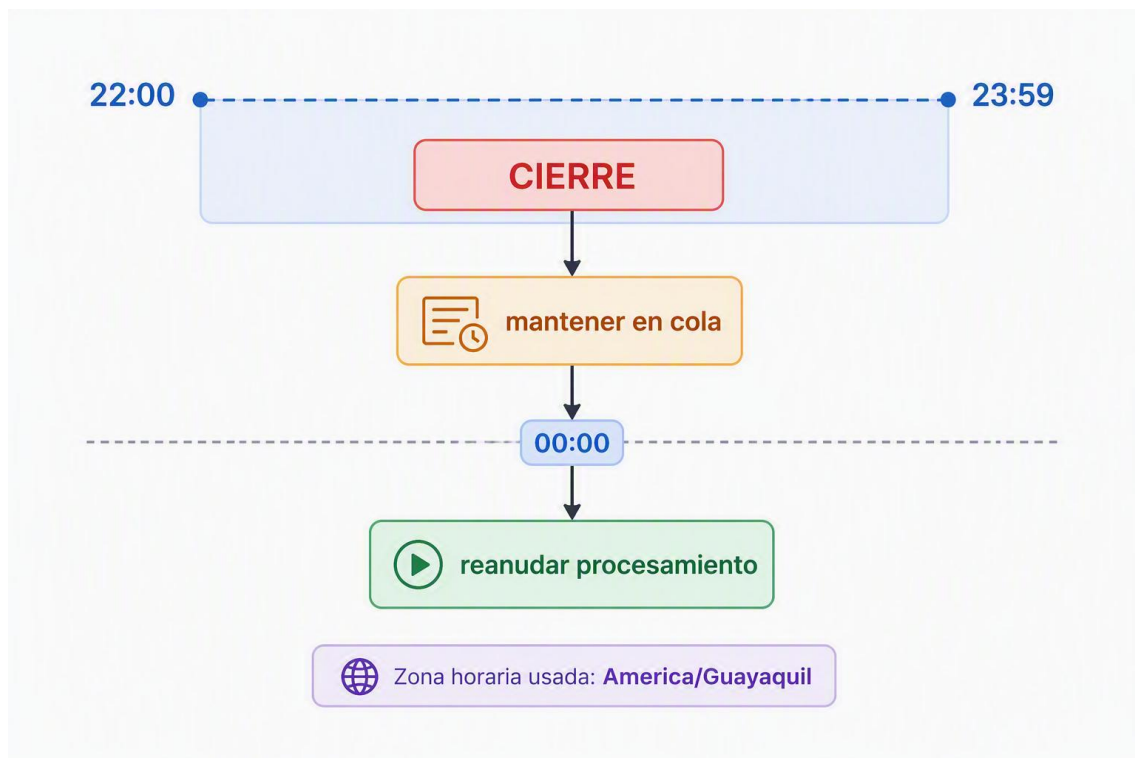


Ilustración 2 - Autoria Propia

11. Reintentos y backoff exponencial

Las comunicaciones con sistemas externos incorporan hasta tres intentos. Entre intentos se aplica backoff exponencial mediante la expresión:

$$\text{retry_base} \times 2^{(\text{intento}-1)}$$

Para la configuración utilizada (retry_base=1), los tiempos de espera son 1 segundo después del primer fallo y 2 segundos después del segundo.

12. Idempotencia

La factura se almacena en facturas_emitidas inmediatamente después de su emisión. Si posteriormente Gestión o Lealtad presentan un fallo, un reprocesamiento puede recuperar la factura ya conocida en lugar de emitir una nueva.

La prueba automatizada correspondiente verificó tres intentos hacia Gestión y una sola llamada hacia Contable.

NOTA: Para la PoC este mecanismo se mantiene en memoria. Una solución productiva debería utilizar persistencia externa.

13. DLQ (cola de mensajes muertos)

Los fallos definitivos del Sistema Contable son enviados a facturacion_dlq, conservando la venta y el detalle del error para análisis o reprocesamiento posterior.

14. Logging estructurado

```
{  
  "timestamp": "2026-08-17T00:59:03.755543-05:00",  
  "id_venta": "V-LOG-001",  
  "sistema_destino": "SISTEMA_CONTABLE",  
  "estado": "OK",  
  "detalle_error": null  
}
```

Para el logging se generaron 3 tipos de estados:

- OK
- FALLIDO
- ENCOLADO

15. Estrategia de rollback y compensación

Debido a que la integración involucra sistemas independientes, no se utiliza una transacción distribuida tradicional. La recuperación se basa en reintentos, idempotencia y compensación.

Si Contable falla antes de emitir la factura, no existe operación que revertir y la solicitud se reintenta. Si se agotan los intentos, el evento se conserva en la DLQ.

Si Contable ya emitió la factura pero falla la notificación a Gestión, no debe anularse ni emitirse nuevamente la factura. La acción de recuperación consiste en reintentar únicamente la notificación pendiente.

De manera equivalente, si Gestión fue actualizado, pero Lealtad falla, la factura permanece válida y se reintenta únicamente la acreditación de puntos.

Si una operación de negocio requiriese anular posteriormente una factura ya emitida, debería implementarse una operación compensatoria explícita contra el Sistema Contable y, si los puntos ya fueron acreditados, una compensación equivalente en Lealtad.

16. Pruebas automatizadas

Prueba	Objetivo	Resultado
Venta exitosa	Verificar flujo Contable → Gestión → Lealtad	PASSED
Cierre nocturno	Validar regla horaria 22:00–23:59	PASSED
Fallo Contable	Comprobar reintentos y recuperación	PASSED
Fallo Gestión	Comprobar reintentos sin duplicar factura	PASSED
Payload inválido	Comprobar rechazo HTTP 400	PASSED

Salida en consola:

```
collected 5 items

test_venta_procesada_exitosamente PASSED      [20%]
test_horario_cierre PASSED                    [40%]
test_fallo_contable_con_reintento PASSED      [60%]
test_fallo_gestion_sin_duplicar_factura PASSED [80%]
test_payload_invalidado PASSED                [100%]

5 passed in 7.24s
```

Ilustración 3 - autoría propia

17. Resultados

La PoC permitió validar el flujo completo de integración y los principales escenarios de resiliencia. En condiciones normales, una venta es recibida por el Orchestrator, publicada en RabbitMQ, procesada por el Worker, facturada por el Sistema Contable y posteriormente notificada a Gestión y Lealtad.

Las pruebas de fallo demostraron que los errores temporales pueden recuperarse mediante reintentos con backoff exponencial. Asimismo, el escenario de indisponibilidad temporal de Gestión confirmó que los reintentos de notificación no generan una segunda factura.

La ejecución automatizada final obtuvo cinco pruebas satisfactorias (5 passed).

18. Limitaciones

- Idempotencia en memoria.
- Mocks sin base de datos.
- Contadores de pruebas en memoria.
- Sin autenticación/autorización entre servicios.
- Sin plataforma centralizada de observabilidad.
- DLQ sin proceso automático de reprocesamiento.
- La prueba automatizada de cierre valida la regla horaria, no simula el transcurso completo de 22:00 a 00:00.

19. Conclusiones

La solución desarrollada demuestra que una arquitectura desacoplada basada en mensajería permite mejorar la resiliencia del proceso de facturación frente a una integración exclusivamente síncrona.

RabbitMQ permite conservar y desacoplar los eventos de venta, mientras que el Worker centraliza las reglas relacionadas con cierre contable, reintentos y coordinación entre sistemas.

La utilización del patrón Adapter reduce el acoplamiento con los sistemas externos y facilita su futura sustitución por implementaciones reales.

Los mecanismos de backoff exponencial, idempotencia, DLQ y logging estructurado permiten abordar los principales riesgos identificados durante la fase de análisis.

Finalmente, la ejecución satisfactoria de las cinco pruebas automatizadas demuestra el funcionamiento de los escenarios principales requeridos por la PoC y proporciona un mecanismo reproducible para validar la solución mediante Docker Compose.